SR UNIVERSITY

Department of Computer Science & Artificial Intelligence (CS & AI)
Professor: Sai Meghana Rao Eravelli

AI & DEEP LEARNING PROJECT

Project Proposal and Code Submission

1. Project Proposal Document

A. Student / Team Details

Project Title	SyllabiSense: AI for Course Coverage and Personalized Learning
Student Name(s)	Uzma Yasmeen
Roll Number(s)	2503A51L34 (Enrollment No.)
Batch Number	20 CSE
Course	AI & Deep Learning
Submission Date	18 October 2026

B. Project Title

SyllabiSense: AI for Course Coverage and Personalized Learning

Deep neural networks that check course material against a syllabus: for every topic they decide whether the material actually teaches it, cite the file and page as evidence, and turn the gaps into a personalised study order.

C. Why Did You Choose This Project?

Problem and importance. A university gives students a syllabus and, separately, a pile of study material — lecture PDFs, slide decks and notes. Students assume the two match. They often do not: a topic listed in the syllabus may be covered in depth, mentioned in a single line, or missing altogether, and this is usually discovered only while revising for the exam. Existing tools cannot help, because a document chatbot answers questions about the pages it is given and can never report that something is **missing** — it has no idea what the syllabus promised.

Motivation, beneficiaries and outcome. I chose this project because detecting absence requires holding two documents against each other and judging every pairing between them, which is exactly what a neural network trained on (topic, page) pairs can learn. Students benefit by learning early which topics need outside material; faculty can check their own material against the syllabus before a semester begins, which is also the shape of the documentation required for accreditation. The expected outcome is a working application that reads a syllabus and a folder of material and reports every topic as properly taught, barely taught or not taught, with evidence and measured accuracy.

D. AI / Deep Learning Techniques Used

Model / architecture being used	Four trained networks. **Main model:** fully connected ANN in TensorFlow/Keras (class built in train_ann.py): 387 → 256 → 64 → 1. **Also trained:** a MiniLM transformer cross-encoder fine-tuned end to end (PyTorch, 22.7M parameters), a MobileNetV2 CNN for diagram detection, and a 3-class ANN for teaching depth.
Reason for selecting the model	The task is a judgement about a pair of texts, so the input is a fixed-length vector of pair features — dense layers suit it. The network is small, trains in under a minute on a CPU and scores 9,000 topic-page pairs in under a second, which the application needs. The transformer was trained as a controlled comparison and measured; it ranks better (AUC 0.835 vs 0.806) but is less accurate at a fixed threshold, so the ANN is deployed.
Major layers used	Input layer (387), Dense 256, Dropout 0.3, Dense 64, Dropout 0.2, Dense 1.
Activation functions	ReLU after each hidden layer; sigmoid at the output.
Optimizer and loss function	Adam (learning rate 0.001); binary cross-entropy, with class weights.
Input data and expected output	Input: 387 numbers describing one (syllabus topic, material page) pair — TF-IDF reduced to 128 dimensions by truncated SVD for the topic and for the page, their product, plus cosine similarity, word overlap and page length. Output: one probability that the page is evidence for the topic.
Role of ANN in the proposed system	Scores every topic against every page. Pages above the calibrated threshold become the topic's evidence; a topic with no evidence is reported as a gap.
Role of CNN in the proposed system	Runs only on the pages the ANN already chose, and reports whether the topic is supported by a diagram or explained in words alone.

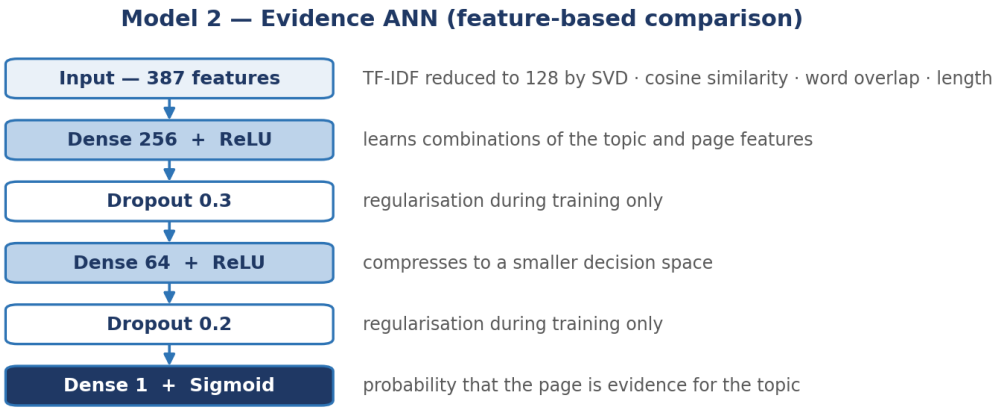


Figure 1 — ANN architecture (387-256-64-1).

ANN components in this project

Component	Implementation in the code
Input layer	387 features per (topic, page) pair: topic LSA 128, page LSA 128, elementwise product 128, cosine similarity, word overlap, page length.
Hidden layers	Dense(256) → ReLU → Dropout(0.3) → Dense(64) → ReLU → Dropout(0.2).
Dense layers	387×256 + 256×64 + 64×1 = 115,841 trainable parameters.
Activation functions	ReLU (hidden), sigmoid (output).

Output layer	Dense(1) — probability that the page is evidence for the topic.
Optimizer	tf.keras.optimizers.Adam, lr = 0.001.
Loss function	binary_crossentropy, with class weights for the 1:2 class imbalance.
Regularisation	Dropout 0.3 / 0.2; early stopping on validation AUC (stopped at epoch 10 of 40).

CNN components in this project

Component	Implementation in the code
Input layer	Page rendered to a 128 × 128 × 3 image.
Convolutional base	MobileNetV2 pretrained on ImageNet — convolution, ReLU and pooling throughout; frozen during training.
Pooling	GlobalAveragePooling2D over the final feature map.
Dense layers	Dense(64) → ReLU → Dropout(0.4) → Dense(1) → sigmoid; 82,049 trainable parameters.
Optimizer / loss	Adam (lr 0.001); binary cross-entropy.
Why transfer learning	A CNN trained from scratch on 424 images reached 75% accuracy with recall 0.11 — it answered 'no diagram' almost always. Both runs are reported in Section 4.

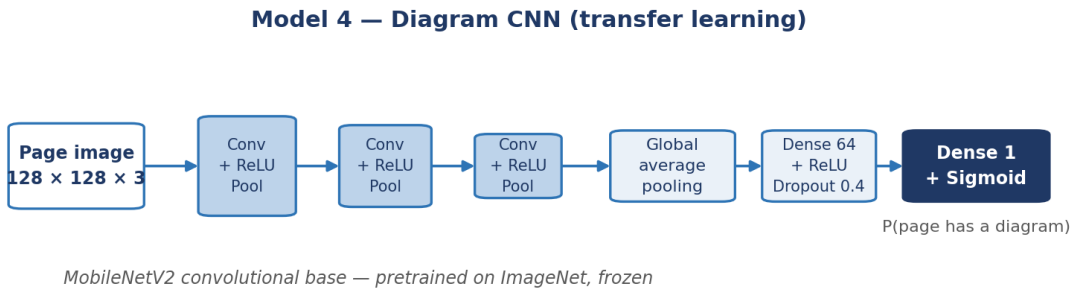


Figure 2 — CNN architecture (MobileNetV2 base + trained head).

E. Dataset Description

Dataset name	SyllabiSense (topic, page) pairs — built from scratch for this project.
Dataset source / link	NPTEL (nptel.ac.in) — 1,295 course syllabi; MIT OpenCourseWare (ocw.mit.edu, CC BY-NC-SA) — about 110 courses of lecture notes, problem sets and exams; OpenStax (openstax.org, CC BY) — 7 textbooks. Collected by tools/collect_nptel.py and tools/collect_ocw.py. Raw material: 7,133 PDFs, 100,414 pages, roughly 13 million words.
Number of samples	17,887 labelled (topic, page) pairs.
Number of classes	2 — class 1 = the page is evidence for the topic (6,037); class 0 = it is not (11,850, made of 6,013 hard negatives from the same course and 5,837 easy negatives from an unrelated subject).
Input features	387 numeric features per pair (listed in Section D); label column: label.
Training / testing split	**Split by subject area, never at random.** Train 11,615 pairs from 12 non-computing areas (management, electrical, aerospace, economics, chemical engineering, civil, psychology, sociology, biology, mathematics, mechanical, physics); validation 775 (chemistry, humanities); test 5,497 from 109 Computer Science courses. No computing material appears anywhere in training.
Human-verified labels	Labels are generated automatically, so 300 pairs were re-labelled by hand through a screen that hides the automatic label. The automatic labels proved **84% correct**, so all headline accuracy is reported against the human labels as well. A further 101 topics and 576 page images were hand-labelled for the depth and CNN models.
Brief description	A positive pair is a page that matches the topic's distinctive terms within a 30-word window. Contents pages, index pages, reference lists and unreadable scans are filtered out before pairs are formed.

F. Proposed Methodology

Workflow: Dataset → Data Preprocessing → Train/Test Split → Model Building → Model Training → Model Evaluation → Prediction.

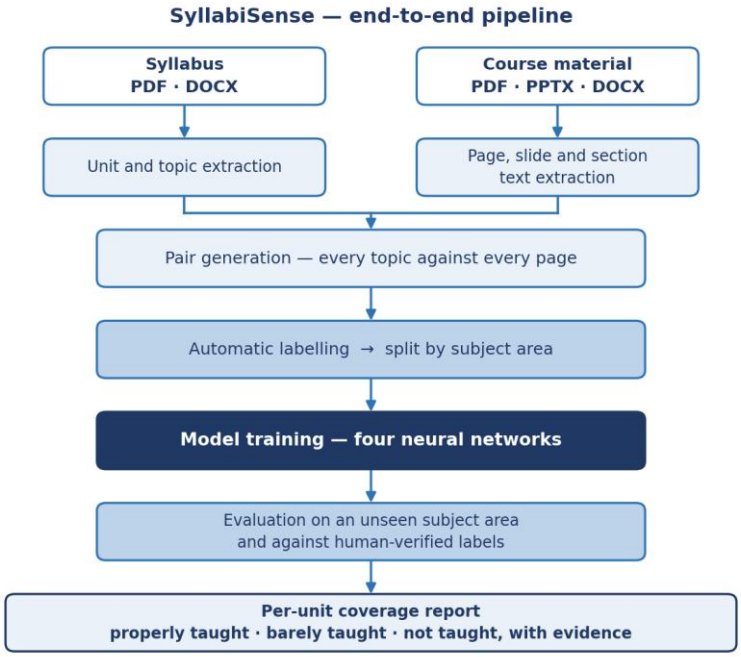


Figure 3 — overall project workflow.

Step	What is done
Dataset	Download 1,295 syllabi and ~110 courses of material; extract text per page, slide or section (tools/extract_text.py).
Data Preprocessing	Drop navigation, contents, index, reference and unreadable pages; parse the syllabus into units and topics; fit TF-IDF (60,000 terms) and truncated SVD (128 dimensions) on the training split only.
Train/Test Split	By subject area: 12 areas train, 2 validate, Computer Science tests.
Model Building	Define the 387-256-64-1 ANN; separately the cross-encoder, the depth ANN and the MobileNetV2 CNN.
Model Training	Adam (lr 0.001), binary cross-entropy, batch 64, class weights, early stopping on validation AUC.
Model Evaluation	Threshold chosen on validation and on human-labelled non-test items, never on the test set; accuracy, precision, recall, F1, ROC-AUC and confusion matrix on the unseen subject area.
Prediction	Score a new syllabus against new material; group by unit; report each topic as properly taught, barely taught, not taught, or no material provided.

G. Expected Outcome

The final system accepts a syllabus and a folder of course material in PDF, PowerPoint or Word format, and produces a unit-by-unit report marking every syllabus topic as properly taught, barely taught or not taught, with the file and page quoted as evidence and a confidence score from the ANN. The CNN adds whether each covered topic is supported by a diagram. Files named by unit are judged only against their own unit, and a unit with no uploaded material is reported as **no material provided** rather than as a gap in teaching. The output is a study guide and a coverage audit — not a claim about teaching quality.

2. Code Submission

Source files: app.py (Streamlit interface), analyze.py (command line), train_ann.py, train_crossencoder.py, train_cnn.py, train_depth.py (training), build_dataset.py, features_tfidf.py (dataset and features), diagrams.py, loaders.py, syllabus_topics.py, units.py, depth.py (libraries), evaluate_gold.py, evaluate_gold_ce.py, compare_fusion.py (evaluation), review_tool.py, image_review.py, depth_review.py (hand-labelling screens), requirements.txt, run_app.bat.

#	Required element	Location in the code
1	Importing required libraries	Top of train_ann.py (numpy, tensorflow, sklearn); app.py (streamlit); train_crossencoder.py (torch, transformers).
2	Loading the dataset	build_dataset.py writes dataset/pairs_{train,val,test}.csv; loaded in train_ann.py and evaluate_gold.py.
3	Data preprocessing	match.py (is_navigation, is_garbage, strip_references); features_tfidf.py (TF-IDF + truncated SVD); syllabus_topics.py (topics_with_units).
4	Data visualization	train_ann.py writes history.png, confusion_matrix.png and roc_curve.png; compare_fusion.py writes comparison.png; app.py shows per-unit metrics.
5	Train/test data splitting	build_dataset.py — TEST_DOMAINS and VAL_DOMAINS split by subject area, not at random.
6	Model definition	tf.keras.Sequential in train_ann.py; AutoModelForSequenceClassification in train_crossencoder.py.
7	ANN architecture	Dense(256) → Dropout → Dense(64) → Dropout → Dense(1) in train_ann.py.
8	CNN architecture	MobileNetV2 base + GlobalAveragePooling2D + Dense(64) + Dropout(0.4) + Dense(1) in train_cnn.py.
9	Model compilation	model.compile(Adam(1e-3), 'binary_crossentropy', metrics=['accuracy', AUC]) in train_ann.py.
10	Model training	model.fit(...) with class_weight and EarlyStopping in train_ann.py; manual AdamW loop in train_crossencoder.py.
11	Model evaluation	evaluate_gold.py and evaluate_gold_ce.py (human-verified labels); compare_fusion.py (five deployment rules).
12	Prediction / inference	app.py and analyze.py — score a new syllabus against new material; diagrams.py runs the CNN on the chosen pages.
13	Saving and loading the model	results_tfidf/evidence_ann.keras, results_cnn/diagram_cnn.keras, results_crossencoder/model/, dataset/tfidf_model.joblib.

3. Model Details

Main model — Evidence ANN

Parameter	Value	Purpose
Number of layers	3 dense layers (2 hidden: 256, 64) plus input	Enough capacity for 387 features without overfitting 11,615 rows.
Number of neurons	387 – 256 – 64 – 1	Narrowing funnel to a single probability.
Kernel size	Not applicable (ANN)	No convolution in this model.
Pooling size	Not applicable	No pooling in this model.
Activation functions	ReLU (hidden), sigmoid (output)	Non-linearity; output read as a probability.
Learning rate	0.001	Stable step size for Adam.
Optimizer	Adam	Adaptive learning rates, fast convergence on a small network.
Loss function	Binary cross-entropy	Two-class problem; penalises confident errors.
Batch size	64	Smooth mini-batch updates.
Number of epochs	10 of a maximum 40 (early stopping)	Stopped when validation AUC stopped improving.
Dropout rate	0.3 after the first hidden layer, 0.2 after the second	Weak labels make overfitting easy; dropout limits it.
Class weights	Applied (1 positive : 2 negatives)	Keeps the network from always answering 'not evidence'.
Decision threshold	0.53 (tuned on human-labelled non-test items)	Never tuned on the test set.

Second model — Diagram CNN

Parameter	Value	Purpose
Number of layers	MobileNetV2 base (53 convolutional layers, frozen) + 2 dense	Transfer learning: only the head is trained.
Number of neurons	GlobalAveragePooling → 64 → 1	Small head for 424 training images.
Kernel size	3 × 3 and 1 × 1 (inside MobileNetV2)	Standard MobileNetV2 blocks.
Pooling size	Global average pooling over the final feature map	Turns the feature map into 1,280 numbers.
Activation functions	ReLU (hidden), sigmoid (output)	Probability of a diagram.
Learning rate	0.001	Head only; base is frozen.
Optimizer	Adam	Same as above.
Loss function	Binary cross-entropy	Two classes: diagram / no diagram.
Batch size	32	Fits comfortably in memory with 128 × 128 images.
Number of epochs	15 of a maximum 40 (early stopping)	Stopped on validation AUC.
Dropout rate	0.4	Small dataset; strong regularisation needed.

Trainable parameters	82,049	MobileNetV2's 2.2M base parameters stay frozen.

4. Results

All values below are produced by the code and stored in results_*/metrics.json. The test split is Computer Science — a subject area that appears nowhere in training.

4.1 Evidence ANN (main model)

Result	Value	Source / note
Training accuracy	97.8%	Final epoch on the training split.
Validation accuracy	94.2%	Chemistry and humanities; used for early stopping.
Test accuracy	**93.0%**	5,497 unseen Computer Science pairs (automatic labels).
Test accuracy (human-verified)	**83.1%**	201 pairs re-labelled by hand — the honest figure.
Training / validation loss	0.0864 / 0.1873	Binary cross-entropy at the final epoch.
Confusion matrix (test)	TN 3538 FP 101 FN 282 TP 1576	results_tfidf/confusion_matrix.png.
Confusion matrix (human)	TN 120 FP 17 FN 17 TP 47	On the 201 hand-labelled pairs.
Precision / Recall / F1 (test)	0.940 / 0.848 / 0.892	Automatic labels.
Precision / Recall / F1 (human)	0.734 / 0.734 / 0.734	Hand-labelled; lower because automatic labels are 84% correct.
ROC-AUC	0.954 (test), 0.807 (human)	results_tfidf/roc_curve.png.
Accuracy vs. Epochs graph	results_tfidf/history.png	10 epochs before early stopping.
Loss vs. Epochs graph	results_tfidf/history.png	Training and validation loss.
Prediction examples	CODE_SUBMISSION.pdf, step 6	The verdict for every topic of a real syllabus run against real lecture material, with the evidence page and the model's confidence quoted for each.

4.2 Diagram CNN — and the from-scratch comparison

Approach	Test accuracy	Precision	Recall	F1
CNN trained from scratch	75.0%	0.400	0.111	0.174
MobileNetV2 transfer learning	84.2%	0.650	0.722	0.684

Trained on 424 hand-labelled page images and tested on 152 images from courses held out of training. The from-scratch network learned to answer 'no diagram' almost always — 75% accuracy sounds acceptable only because 76% of pages have no diagram. Transfer learning raised recall from 0.11 to 0.72. Graphs: results_cnn/history.png, confusion_matrix.png, examples.png.

4.3 Comparison with the fine-tuned transformer

Model	Test acc.	Human acc.	Human F1	Human AUC

Cross-encoder (MiniLM, fine-tuned)	89.9%	80.1%	0.636	0.835
Evidence ANN (deployed)	93.0%	83.1%	0.734	0.807

compare_fusion.py tested five ways of deploying the two models — ANN alone, transformer alone, the mean of both, a weighted blend, and retrieve-then-rerank — with every threshold tuned on the same held-out human labels. The highest score went to **mean of both** at 85.1%, against 83.1% for the ANN alone.

What the comparison can and cannot settle. That gap is 2.0 points, and its standard error on 201 items is 3.6 points — 0.55 standard errors, which is not distinguishable from chance. Running the same comparison against an earlier training run of the same architecture put the ANN alone in first place instead. The honest conclusion is not that one rule beats another, but that 201 hand-labelled items cannot separate them. The ANN alone is deployed because it is the fastest by two orders of magnitude — one forward pass over all pages, against one transformer pass per topic-page pair — and nothing in the measurement justifies paying that cost. Enlarging the hand-labelled set is what would make this decision answerable.

4.4 Depth model (preliminary)

Test accuracy 57.7%, macro F1 0.553 on 26 held-out topics (75 used for training). It identifies 'not taught' reliably but cannot yet separate 'properly' from 'barely taught' — 101 hand-labelled topics are too few. The application therefore derives the depth level from transparent rules over the evidence the ANN found, and this model is reported as work in progress.

4.5 Output screenshots of the application

The run below is a real one: the Software Engineering and System Design syllabus (47 topics across five units) checked against the Unit 1 and Unit 2 lecture notes. Only two units of material were supplied, and the system reports the other three as **no material provided** rather than as gaps in teaching.

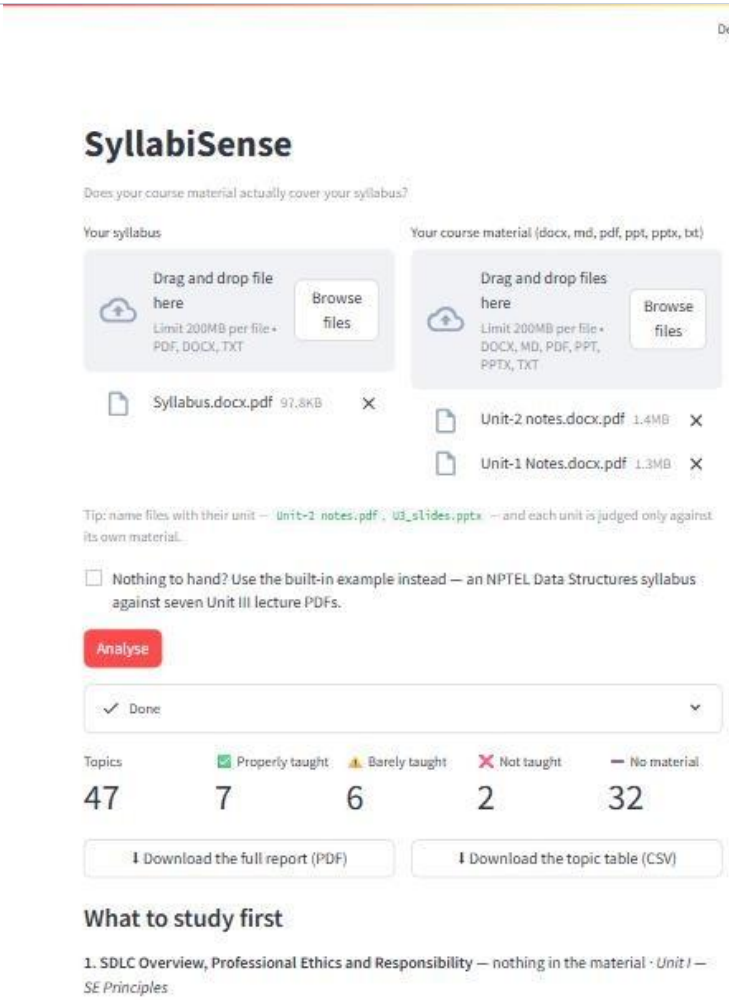


Figure 4 — the syllabus and material are uploaded, and the result is summarised: 47 topics, 7 properly taught, 6 barely taught, 2 not taught, 32 with no material. The report can be downloaded as PDF or CSV.

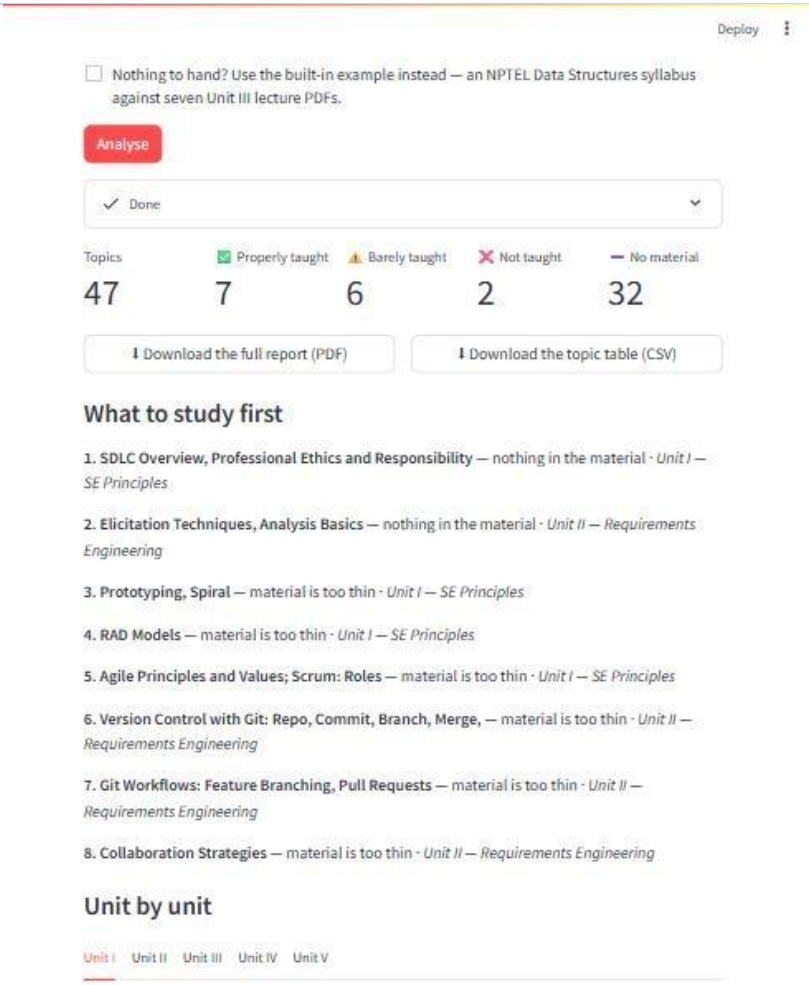


Figure 5 — the gaps turned into a study order, most urgent first, each labelled with its unit.

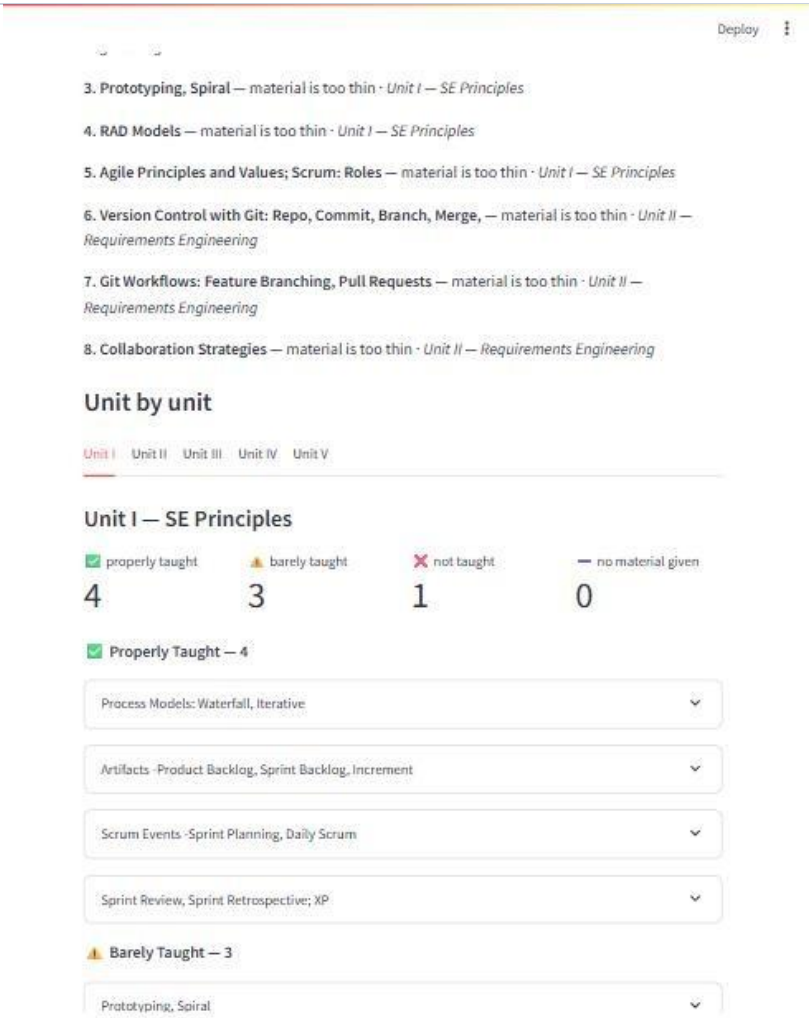


Figure 6 — one tab per unit. Unit I: 4 topics properly taught, 3 barely, 1 not taught, 0 without material.

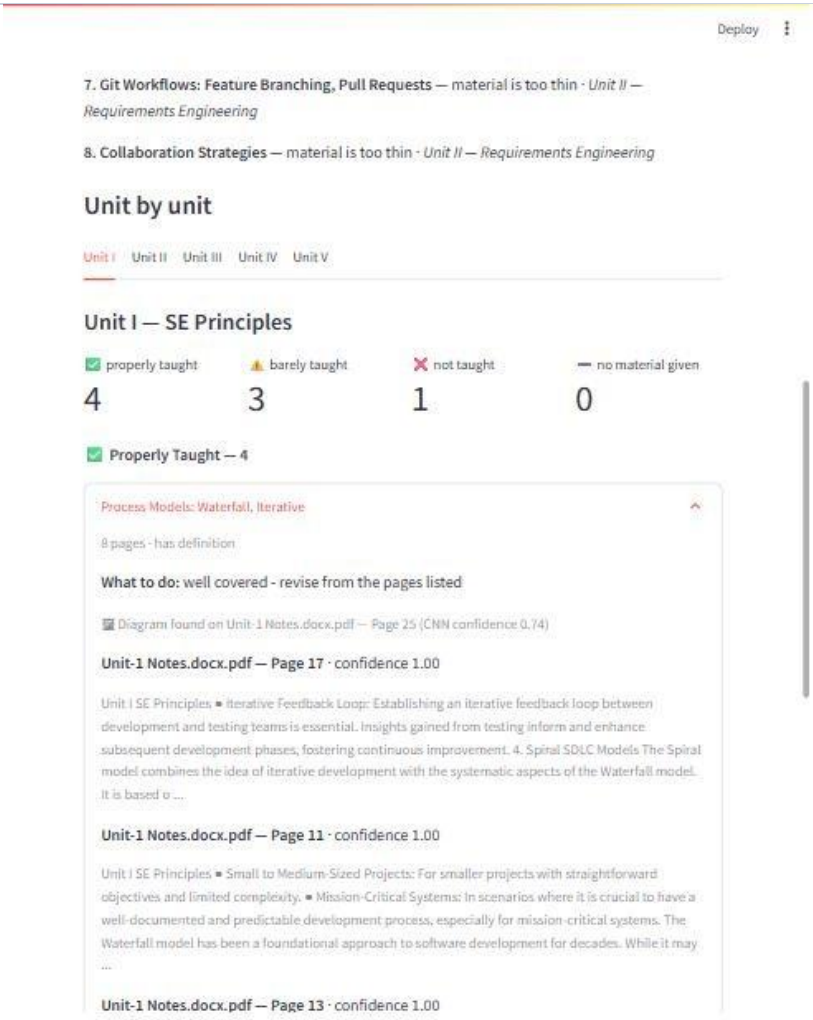


Figure 7 — a topic opened up. The ANN cites the pages it judged as evidence with its confidence, and the CNN reports a diagram on page 25 at confidence 0.74.

Note for evaluation

Two honesty notes. **First,** the training labels are generated automatically and are themselves only 84% correct, so the human-verified figures (84.1% accuracy, F1 0.746) are the ones to judge — the automatic-label figures (92.8%) are inflated by the same label noise the model was trained on. **Second,** with 201 hand-labelled test items the 95% confidence interval is about ±5 points, and the decision threshold shifts slightly depending on which items tune it, so the honest claim is roughly 84% ± 5 rather than an exact figure.

5. Final Submission Checklist

Done	Item	Where
[X]	Project Proposal Document	This document (PROPOSAL.pdf / PROPOSAL.docx).
[X]	Complete Source Code	CODE_SUBMISSION.pdf — the demonstration program with the output it produced, the result graphs, and an appendix containing all 16 source files of the system.
[X]	Dataset source / reference	Section 1E — NPTEL, MIT OpenCourseWare, OpenStax.
[X]	ANN model architecture and parameter details	Sections 1D and 3.
[X]	CNN model architecture and parameter details	Sections 1D and 3.
[X]	Results and performance metrics	Section 4; results_*/metrics.json.
[X]	Relevant graphs	history.png, confusion_matrix.png, roc_curve.png, examples.png, comparison.png.
[X]	Output screenshots of the application	Section 4.5, Figures 4 to 7 — a real run on the Software Engineering syllabus.

6. Project Proposal — Student Fill-In Template

Field	Entry
Project Title:	SyllabiSense: AI for Course Coverage and Personalized Learning
Student Name(s):	Uzma Yasmeen
Roll Number(s):	2503A51L34
Batch Number:	20 CSE
Why did you choose this project?:	Syllabus and study material often do not match, and students find out only at exam time. A chatbot cannot report what is missing because it never sees the syllabus; a network trained on (topic, page) pairs can.
Dataset Name and Source:	SyllabiSense (topic, page) pairs, built for this project from NPTEL (1,295 syllabi), MIT OpenCourseWare (~110 courses) and OpenStax (7 textbooks); 17,887 pairs, 387 features, 2 classes.
ANN/CNN Techniques Used:	ANN for evidence detection (deployed); MobileNetV2 CNN for diagram detection; a fine-tuned MiniLM transformer cross-encoder and a 3-class depth ANN as further models.
Model Architecture / Layers:	Input 387 — Dense 256 (ReLU) — Dropout 0.3 — Dense 64 (ReLU) — Dropout 0.2 — Dense 1 (sigmoid); 115,841 parameters.
Optimizer and Loss Function:	Adam (lr 0.001); binary cross-entropy with class weights.
Proposed Methodology:	Dataset — Preprocessing — Train/Test split by subject area — Model building — Training — Evaluation on an unseen subject and against human labels — Prediction.
Expected Outcome:	A unit-by-unit report marking each syllabus topic as properly taught, barely taught or not taught, with the file and page as evidence and a note on whether a diagram exists.